

Rapport de Projet : Space Tourism

Informations Générales

Projet : Space Tourism - Site Web et Back-Office de Gestion

Période : Novembre 2025

Technologies : Laravel 12, PHP 8.x, MySQL, Tailwind CSS, Vite, Blade

Dépôt GitHub : <https://github.com/miloudi74380-cmd/space-tourism>

Table des Matières

1. [Objectifs du Projet](#)
 2. [Architecture et Choix Techniques](#)
 3. [Réalisations par Partie](#)
 4. [Fonctionnalités Développées](#)
 5. [Sécurité et Validation](#)
 6. [Tests et Qualité](#)
 7. [Difficultés Rencontrées et Solutions](#)
 8. [Conclusion et Perspectives](#)
-

1. Objectifs du Projet

1.1 Contexte

Développer un site web complet pour une agence de tourisme spatial fictive avec :

- Un site public responsive et multilingue
- Un back-office complet pour la gestion des contenus
- Un système d'authentification et de gestion des utilisateurs
- Une architecture MVC respectant les bonnes pratiques Laravel

1.2 Objectifs Pédagogiques

- Maîtriser le framework Laravel (routing, controllers, models, migrations)
 - Implémenter un système d'authentification et d'autorisation
 - Créer une interface responsive avec Tailwind CSS
 - Gérer l'internationalisation (i18n) d'une application web
 - Utiliser Git pour le versionnement et GitHub pour le déploiement
 - Structurer un projet selon l'architecture MVC
 - Implémenter des tests unitaires et fonctionnels
-

2. Architecture et Choix Techniques

2.1 Stack Technique

Backend

- **Laravel 12.39.0** : Framework PHP moderne et robuste
- **PHP 8.x** : Dernière version stable avec typage strict
- **MySQL** : Base de données relationnelle
- **Spatie Laravel Permission** : Gestion des rôles et permissions
- **Laravel Breeze** : Scaffolding d'authentification

Frontend

- **Tailwind CSS 3.x** : Framework CSS utility-first
- **Blade** : Moteur de templates Laravel
- **Vite** : Build tool moderne pour les assets
- **JavaScript vanilla** : Interactivité côté client

Outils de Développement

- **Git & GitHub** : Versionnement et collaboration

- **Composer** : Gestionnaire de dépendances PHP
- **NPM** : Gestionnaire de dépendances JavaScript
- **Laravel Artisan** : CLI pour la gestion du projet

2.2 Structure du Projet

```
space-tourism/
├── app/
│   ├── Http/Controllers/      # Contrôleurs publics et admin
│   ├── Models/                # Modèles Eloquent
│   └── View/Components/       # Composants Blade
├── database/
│   ├── migrations/            # Migrations de base de données
│   └── seeders/               # Données initiales
├── resources/
│   ├── views/                 # Templates Blade
│   ├── js/                    # Scripts JavaScript
│   └── css/                    # Styles CSS
├── routes/
│   └── web.php                 # Définition des routes
└── public/
    └── assets/                 # Images et ressources statiques
```

2.3 Choix Architecturaux

Patron MVC (Model-View-Controller)

- **Models** : Planet, Crew, Technology, User
- **Views** : Templates Blade séparés (public/admin)
- **Controllers** : Logique métier séparée par domaine

Séparation des Responsabilités

- Controllers publics : Affichage du site
- Controllers Admin : Gestion CRUD
- Middleware : Authentification et autorisation
- Seeders : Données de test reproductibles

Internationalisation (i18n)

- Fichiers de traduction FR/EN dans lang/
- Colonnes bilingues dans la base de données
- Helper methods dans les models pour récupération locale-aware

3. Réalisations par Partie

Partie 1 : Initialisation et Page d'Accueil

Objectif : Mettre en place la structure du projet et créer la page d'accueil

Réalisations

- Installation et configuration de Laravel 12
- Configuration de Tailwind CSS et Vite
- Création du layout responsive principal (layouts/app.blade.php)
- Page d'accueil avec design spatial et navigation
- Backgrounds responsifs (mobile/tablet/desktop)
- Intégration des polices Google Fonts (Barlow Condensed, Bellefair)

Fichiers Créés

- resources/views/home.blade.php
- resources/views/layouts/app.blade.php
- resources/views/components/navigation.blade.php
- resources/css/app.css
- tailwind.config.js

Partie 2 : Internationalisation (i18n)

Objectif : Rendre le site multilingue (Français/Anglais)

Réalisations

- Configuration du système de traduction Laravel
- Création des fichiers de langue `lang/fr/` et `lang/en/`
- Contrôleur `LanguageController` pour le changement de langue
- Middleware de détection de locale
- Composant de sélection de langue dans la navigation
- Sauvegarde de la préférence en session

Fichiers Créés

- `app/Http/Controllers/LanguageController.php`
- `lang/fr/*.php` (home, destination, crew, technology, navigation)
- `lang/en/*.php` (traductions anglaises)
- Route `/language/{locale}`

Clés de Traduction

- Navigation, titres, descriptions
 - Contenu des 4 pages principales
 - Données dynamiques (planètes, équipage, technologies)
-

Partie 3 : Pages Publiques et Tests

Objectif : Créer les 3 pages restantes et tester les routes

Réalisations

Page Destination

- Affichage de 4 planètes (Moon, Mars, Europa, Titan)
- Navigation par onglets avec JavaScript
- Changement dynamique des images et descriptions
- Données i18n depuis les fichiers de langue

Page Crew

- Présentation de l'équipage (4 membres)
- Navigation par points (dots)
- Changement dynamique du contenu
- Affichage des rôles et biographies

Page Technology

- 3 technologies spatiales (Launch vehicle, Spaceport, Space capsule)
- Navigation numérotée (1, 2, 3)
- Images landscape (mobile/tablet) et portrait (desktop)
- Responsive design avancé

Tests Unitaires

- Tests des routes publiques (`/`, `/destination`, `/crew`, `/technology`)
- Tests de changement de langue
- Vérification des réponses HTTP 200
- Tests d'intégration du système i18n

Fichiers Créés

- `resources/views/destination.blade.php`
- `resources/views/crew.blade.php`
- `resources/views/technology.blade.php`
- `resources/js/destination.js`
- `resources/js/crew.js`
- `resources/js/technology.js`
- `tests/Feature/RouteTest.php`

Scripts JavaScript

- Gestion de la navigation entre les onglets
- Changement dynamique du DOM
- Support de l'internationalisation côté client
- Gestion des états actifs/inactifs

Partie 4 : Back-Office - CRUD Planètes

Objectif : Créer le système d'administration pour gérer les planètes

Réalisations

Base de Données

- Migration `create_planets_table` avec colonnes bilingues
- Modèle `Planet` avec méthodes helper `getName()` et `getDescription()`
- Seeder avec les 4 planètes initiales

Authentification

- Installation de Laravel Breeze
- Installation de Spatie Laravel Permission
- Création du rôle "admin" avec permissions `planets.*`
- Middleware `role:admin` pour protéger les routes

Controller Admin

- `Admin\PlanetController` avec 7 méthodes RESTful
- Validation des formulaires
- Messages de succès/erreur
- Autorisation via `$this->authorize()`

Vues Admin

- Layout admin (`layouts/admin.blade.php`)
- Index : table avec pagination
- Create : formulaire de création
- Edit : formulaire de modification
- Boutons d'actions (Modifier, Supprimer)

Synchronisation

- Adaptation de la page publique `/destination`
- Récupération des planètes depuis la base de données
- Passage des données via `@json()` pour JavaScript
- Support ID numériques au lieu de clés string

Fichiers Créés

- `database/migrations/XXXX_create_planets_table.php`
- `app/Models/Planet.php`
- `database/seeder/PlanetSeeder.php`
- `database/seeder/RoleSeeder.php`
- `app/Http/Controllers/Admin/PlanetController.php`
- `resources/views/admin/planets/index.blade.php`
- `resources/views/admin/planets/create.blade.php`
- `resources/views/admin/planets/edit.blade.php`
- `app/View/Components/AdminLayout.php`

Permissions Créées

```
'planets.view'  
'planets.create'  
'planets.edit'  
'planets.delete'
```

Partie 5 : Back-Office - CRUD Équipage

Objectif : Étendre le back-office avec la gestion de l'équipage

Réalisations

Base de Données

- Migration `create_crew_table` avec champs bilingues (name, role, bio)
- Modèle `Crew` avec helper methods
- Seeder avec les 4 membres d'équipage

Controller et Vues

- `Admin\CrewController` avec CRUD complet
- 3 vues admin (index, create, edit)
- Formulaires avec validation
- Messages flash de confirmation

Synchronisation

- Page publique `/crew` utilise la base de données
- JavaScript adapté pour IDs numériques
- Locale-aware pour name et role

Fichiers Créés

- `database/migrations/XXXX_create_crew_table.php`
- `app/Models/Crew.php`
- `database/seeder/CrewSeeder.php`
- `app/Http/Controllers/Admin/CrewController.php`
- `resources/views/admin/crew/index.blade.php`
- `resources/views/admin/crew/create.blade.php`
- `resources/views/admin/crew/edit.blade.php`

Permissions Ajoutées

```
'crew.view'  
'crew.create'  
'crew.edit'  
'crew.delete'
```

Partie 6 : Back-Office - CRUD Technologies

Objectif : Compléter le back-office avec la gestion des technologies

Réalisations

Base de Données

- Migration `create_technologies_table`
- Support de 2 images (landscape + portrait)
- Modèle `Technology` avec méthodes `getName()` et `getDescription()`
- Seeder avec 3 technologies

Controller et Vues

- `Admin\TechnologyController` avec CRUD complet
- Vues admin avec champs pour les 2 types d'images
- Validation des chemins d'images

Synchronisation

- Page publique `/technology` dynamique
- JavaScript mis à jour pour gérer les IDs de BDD
- Support des images landscape/portrait

Fichiers Créés

- `database/migrations/XXXX_create_technologies_table.php`
- `app/Models/Technology.php`
- `database/seeder/TechnologySeeder.php`
- `app/Http/Controllers/Admin/TechnologyController.php`
- `app/Http/Controllers/TechnologyController.php`
- `resources/views/admin/technologies/index.blade.php`

- `resources/views/admin/technologies/create.blade.php`
- `resources/views/admin/technologies/edit.blade.php`

Permissions Ajoutées

```
'technologies.view'  
'technologies.create'  
'technologies.edit'  
'technologies.delete'
```

Partie 7 : Gestion des Utilisateurs et Rôles

Objectif : Créer un système complet de gestion des utilisateurs avec rôles granulaires

Réalisations

Système de Rôles

- **Administrateur** : Accès complet (toutes permissions)
- **Gestionnaire Planètes** : CRUD Planètes uniquement
- **Gestionnaire Équipage** : CRUD Équipage uniquement
- **Gestionnaire Technologies** : CRUD Technologies uniquement

Controller Admin/UserController

- CRUD complet des utilisateurs
- Attribution et modification de rôles via formulaire
- Validation email unique et mots de passe sécurisés
- Protection contre auto-suppression
- Synchronisation des rôles avec `syncRoles()`

Vues Admin

- Index avec badges colorés par rôle
- Create avec sélection de rôle
- Edit avec changement de rôle et mot de passe optionnel
- Descriptions des rôles dans les formulaires

Sécurité

- Permissions `users.*` réservées aux Administrateurs
- Vérification `$user->id !== auth()->id()` pour suppression
- Hash des mots de passe avec `Hash::make()`
- Confirmation de mot de passe obligatoire

Fichiers Créés

- `app/Http/Controllers/Admin/UserController.php`
- `resources/views/admin/users/index.blade.php`
- `resources/views/admin/users/create.blade.php`
- `resources/views/admin/users/edit.blade.php`

Permissions Ajoutées

```
'users.view'  
'users.create'  
'users.edit'  
'users.delete'
```

Rôles Créés

```
'admin' // Accès complet  
'planet_manager' // Planètes uniquement  
'crew_manager' // Équipage uniquement  
'technology_manager' // Technologies uniquement
```

Partie 8 : Finitions et Améliorations

Objectif : Finaliser le projet avec les derniers détails professionnels

Réalisations

Favicon

- Ajout du fichier `favicon-32x32.png` dans `public/`
- Intégration dans tous les layouts (`app`, `admin`, `guest`)
- Affichage sur toutes les pages du site

Intégration

```
<link rel="icon" type="image/png" href="{ { asset('favicon-32x32.png') } }">
```

Fichiers Modifiés

- `public/favicon-32x32.png` (nouveau)
- `resources/views/layouts/app.blade.php`
- `resources/views/layouts/admin.blade.php`
- `resources/views/layouts/guest.blade.php`

4. Fonctionnalités Développées

4.1 Site Public (Front-End)

Pages

- **Accueil** : Présentation du tourisme spatial
- **Destination** : 4 planètes avec navigation par onglets
- **Crew** : 4 membres d'équipage avec navigation par points
- **Technology** : 3 technologies avec navigation numérotée

Caractéristiques

- Design responsive (mobile/tablet/desktop)
- Backgrounds différents par breakpoint
- Navigation principale avec indicateur de page active
- Internationalisation FR/EN avec sélecteur de langue
- Données dynamiques depuis la base de données
- JavaScript vanilla pour l'interactivité
- Transitions et animations CSS

4.2 Back-Office (Admin)

Modules de Gestion

1. Planètes

- Liste paginée avec nom FR/EN
- Création avec validation
- Modification des données
- Suppression avec confirmation JavaScript

2. Équipage

- Liste des membres avec rôle
- CRUD complet
- Gestion des biographies bilingues

3. Technologies

- Liste des technologies
- Support de 2 images (landscape/portrait)
- CRUD complet

4. Utilisateurs

- Liste avec badges de rôles colorés
- Création avec attribution de rôle
- Modification de rôle
- Suppression (sauf soi-même)
- Gestion des mots de passe

Caractéristiques Admin

- Interface Tailwind CSS professionnelle
- Navigation admin avec menu
- Messages de succès/erreur (flash messages)
- Pagination sur toutes les listes
- Validation côté serveur
- Protection CSRF
- Autorisation par permissions

4.3 Base de Données

Tables Créées

```
- users (Laravel Breeze)
- roles (Spatie Permission)
- permissions (Spatie Permission)
- model_has_roles (Spatie Permission)
- model_has_permissions (Spatie Permission)
- role_has_permissions (Spatie Permission)
- planets
- crew
- technologies
```

Colonnes Bilingues

Chaque table de contenu possède :

- name_fr / name_en
- description_fr / description_en
- (pour crew) role_fr / role_en, bio_fr / bio_en

Seeders

- RoleSeeder : Création des rôles et permissions
- PlanetSeeder : 4 planètes initiales
- CrewSeeder : 4 membres d'équipage
- TechnologySeeder : 3 technologies
- DatabaseSeeder : Orchestration

4.4 Système d'Authentification

Laravel Breeze

- Login
- Register
- Password Reset
- Email Verification (optionnel)
- Profile Management

Spatie Laravel Permission

- Attribution de rôles aux utilisateurs
- Vérification de permissions dans les controllers
- Middleware `role:admin`
- Méthodes `$this->authorize('permission')`

5. Sécurité et Validation

5.1 Sécurité

Authentification

- Hachage des mots de passe avec bcrypt
- Protection CSRF sur tous les formulaires
- Middleware `auth` pour les routes protégées
- Middleware `role:admin` pour l'administration

Autorisations

- Vérification de permissions avec `authorize()`
- Contrôles au niveau des routes et des controllers
- Protection contre l'escalade de privilèges
- Impossibilité de se supprimer soi-même

Bonnes Pratiques

- Pas de SQL injection grâce à Eloquent ORM
- Échappement automatique des variables Blade
- Validation stricte des entrées utilisateur
- Protection des routes sensibles

5.2 Validation

Règles de Validation

Planètes

```
'name_fr' => 'required|string|max:255'  
'name_en' => 'required|string|max:255'  
'description_fr' => 'required|string'  
'description_en' => 'required|string'  
'image' => 'required|string|max:255'  
'overview_fr' => 'required|string'  
'overview_en' => 'required|string'  
'distance' => 'required|string|max:255'  
'travel_time' => 'required|string|max:255'
```

Équipage

```
'name_fr' => 'required|string|max:255'  
'name_en' => 'required|string|max:255'  
'role_fr' => 'required|string|max:255'  
'role_en' => 'required|string|max:255'  
'bio_fr' => 'required|string'  
'bio_en' => 'required|string'  
'image' => 'required|string|max:255'
```

Technologies

```
'name_fr' => 'required|string|max:255'  
'name_en' => 'required|string|max:255'  
'description_fr' => 'required|string'  
'description_en' => 'required|string'  
'image_landscape' => 'required|string|max:255'  
'image_portrait' => 'required|string|max:255'
```

Utilisateurs

```
'name' => 'required|string|max:255'  
'email' => 'required|string|email|max:255|unique:users'  
'password' => ['required', 'confirmed', Rules\Password::defaults()]  
'role' => 'required|exists:roles,name'
```

Messages d'Erreur

- Affichage des erreurs sous chaque champ
- Bordure rouge en cas d'erreur (`@error`)
- Messages en français
- Préservation des anciennes valeurs avec `old()`

6. Tests et Qualité

6.1 Tests Unitaires

Tests des Routes (RouteTest.php)

```
- test_home_page_loads()
- test_destination_page_loads()
- test_crew_page_loads()
- test_technology_page_loads()
- test_language_switcher()
```

Tests d'Authentification

- Tests Breeze par défaut
- Vérification des redirections
- Tests des middlewares

6.2 Qualité du Code

Respect des Standards

- PSR-4 pour l'autoloading
- PSR-12 pour le style de code
- Conventions de nommage Laravel
- Commentaires PHPDoc sur les méthodes

Organisation

- Séparation claire des responsabilités
- Contrôlers légers avec logique déléguée aux models
- Réutilisation des composants Blade
- DRY (Don't Repeat Yourself)

Versionnement Git

- Commits clairs et descriptifs
- Messages de commit structurés
- Branches master stable
- Historique Git propre

7. Difficultés Rencontrées et Solutions

7.1 Migration des Données de Traduction vers BDD

Problème

Les données étaient initialement dans les fichiers de langue (`lang/fr/`, `lang/en/`). Il fallait les migrer vers la base de données tout en conservant l'internationalisation.

Solution

- Création de colonnes séparées `_fr` et `_en` dans chaque table
- Helper methods dans les models (`getName()`, `getDescription()`)
- Utilisation de `app()->getLocale()` pour récupérer la langue active
- Adaptation du JavaScript pour utiliser les IDs numériques au lieu des clés string

7.2 Gestion des Images Landscape et Portrait

Problème

La page Technology nécessite 2 images différentes (landscape pour mobile/tablet, portrait pour desktop).

Solution

- Ajout de 2 colonnes `image_landscape` et `image_portrait` dans la table
- Deux champs séparés dans les formulaires admin
- JavaScript qui change les 2 images simultanément
- CSS responsive avec `@media` queries

7.3 Synchronisation JavaScript après Migration BDD

Problème

Le JavaScript utilisait des clés string (`moon`, `mars`) qui ne correspondaient plus aux IDs numériques de la BDD.

Solution

- Passage des données via `@json($collection->keyBy('id'))`

- Modification des boutons pour utiliser `data-planet="{{ $planet->id }}"`
- Adaptation des fonctions JavaScript pour accepter des IDs numériques
- Comparaison avec `==` au lieu de `===` pour tolérer les types `string/number`

7.4 Permissions et Rôles Granulaires

Problème

Besoin de rôles spécifiques avec des permissions limitées (un gestionnaire de planètes ne doit pas accéder aux technologies).

Solution

- Création de 4 rôles distincts dans `RoleSeeder`
- Attribution de permissions spécifiques à chaque rôle
- Vérification `$this->authorize()` dans chaque méthode de controller
- Utilisation de `syncRoles()` pour changer le rôle d'un utilisateur

7.5 Page Blanche Admin après Création

Problème

Page blanche lors de l'accès aux routes admin après création du premier module.

Solution

- Création d'un composant `AdminLayout` au lieu d'utiliser directement le layout
- Utilisation de `<x-admin-layout>` dans les vues
- Séparation claire entre layout public (`app.blade.php`) et admin (`admin.blade.php`)

7.6 Tests qui Échouaient après i18n

Problème

Les tests des routes échouaient après l'ajout de l'internationalisation car la session n'avait pas de locale définie.

Solution

- Ajout de `withSession(['locale' => 'en'])` dans les tests
- Vérification de l'existence de `session('locale')` avec fallback sur `config('app.locale')`
- Tests séparés pour chaque langue

8. Conclusion et Perspectives

8.1 Bilan du Projet

Objectifs Atteints ☒

- ☒ Site public complet avec 4 pages responsive
- ☒ Internationalisation FR/EN fonctionnelle
- ☒ Back-office CRUD pour 3 entités (Planètes, Équipage, Technologies)
- ☒ Système de gestion des utilisateurs et rôles
- ☒ Authentification et autorisation sécurisées
- ☒ Base de données structurée avec migrations et seeders
- ☒ Tests unitaires des routes principales
- ☒ Versionnement Git avec historique propre
- ☒ Design professionnel et responsive
- ☒ Favicon intégré sur toutes les pages

Compétences Développées

- **Laravel** : Routing, Controllers, Models, Migrations, Seeders, Middleware, Validation
- **Eloquent ORM** : Relations, Query Builder, Helper Methods
- **Blade** : Templates, Components, Layouts, Directives
- **Authentification** : Laravel Breeze, Sessions, Password Hashing
- **Autorisations** : Spatie Permission, Roles, Policies
- **Front-End** : Tailwind CSS, JavaScript, Responsive Design
- **Base de Données** : MySQL, Migrations, Relations, Indexation
- **Tests** : PHPUnit, Tests Feature, Assertions HTTP
- **Git** : Commits, Branches, Versionnement, Collaboration

8.2 Points Forts

1. Architecture MVC Respectée

- Séparation claire des responsabilités
- Code modulaire et maintenable
- Réutilisation des composants

2. Sécurité

- Authentification robuste avec Breeze
- Permissions granulaires avec Spatie
- Validation stricte des entrées
- Protection CSRF systématique

3. Internationalisation

- Support FR/EN complet
- Données bilingues en BDD
- Changement de langue en temps réel

4. UX/UI

- Design moderne et professionnel
- Responsive sur tous les devices
- Navigation intuitive
- Feedbacks utilisateur (messages de succès/erreur)

5. Qualité du Code

- Conventions Laravel respectées
- Code commenté et documenté
- Tests unitaires
- Commits Git clairs

8.3 Améliorations Possibles

Court Terme

1. Dashboard avec Statistiques

- Nombre total de planètes, crew, technologies, utilisateurs
- Graphiques avec Chart.js ou ApexCharts
- Activité récente (dernières modifications)

2. Système de Recherche

- Recherche globale dans les tableaux admin
- Filtres par nom, date de création
- Tri par colonnes

3. Pagination Améliorée

- Sélection du nombre d'éléments par page
- Liens de pagination personnalisés

Moyen Terme

4. Upload d'Images Réel

- Remplacement des champs texte par upload de fichiers
- Stockage dans `storage/app/public`
- Génération de miniatures
- Validation des types MIME

5. Logs d'Activité (Audit Trail)

- Enregistrement de toutes les actions CRUD
- Affichage de "Qui a fait Quoi et Quand"
- Historique des modifications par entité

6. Internationalisation de l'Admin

- Traduire l'interface admin en FR/EN
- Fichiers de langue pour les messages
- Sélecteur de langue dans le header admin

Long Terme

7. API REST

- Création d'une API pour consommer les données
- Authentification par tokens (Sanctum)
- Documentation Swagger/OpenAPI

8. Tests Avancés

- Tests d'intégration pour les rôles
- Tests de validation des formulaires
- Tests de sécurité (tentatives d'escalade de privilèges)
- Couverture de code > 80%

9. Performance

- Mise en cache des traductions
- Lazy loading des images
- Minification des assets
- CDN pour les ressources statiques

10. Accessibilité (a11y)

- Support complet ARIA
- Navigation au clavier
- Lecteurs d'écran
- Contraste WCAG AA

8.4 Conclusion Professionnelle

Le projet **Space Tourism** représente une application web complète et professionnelle qui démontre une maîtrise solide du framework Laravel et des bonnes pratiques de développement web moderne.

Réussites Majeures

- Architecture MVC rigoureuse et évolutive
- Sécurité et autorisations robustes
- Internationalisation complète et fonctionnelle
- Interface utilisateur moderne et responsive
- Code maintenable et testé
- Documentation et versionnement Git exemplaires

Défis Relevés

- Migration de données de traduction vers BDD
- Gestion de permissions granulaires
- Synchronisation front-end/back-end
- Gestion d'images multiples (landscape/portrait)
- Tests d'intégration avec i18n

Valeur Ajoutée

Ce projet démontre la capacité à :

- Concevoir et développer une application fullstack
- Travailler avec des technologies modernes
- Respecter les standards de l'industrie
- Documenter et tester son code
- Gérer un projet de A à Z

Le projet **Space Tourism** est prêt pour une présentation professionnelle et constitue un excellent ajout à un portfolio de développeur web.

Annexes

A. Structure Complète de la Base de Données

```

-- Table users
CREATE TABLE users (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  email_verified_at TIMESTAMP NULL,
  password VARCHAR(255) NOT NULL,
  remember_token VARCHAR(100) NULL,
  created_at TIMESTAMP NULL,
  updated_at TIMESTAMP NULL
);

-- Table planets
CREATE TABLE planets (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name_fr VARCHAR(255) NOT NULL,
  name_en VARCHAR(255) NOT NULL,
  description_fr TEXT NOT NULL,
  description_en TEXT NOT NULL,
  image VARCHAR(255) NOT NULL,
  overview_fr TEXT NOT NULL,
  overview_en TEXT NOT NULL,
  distance VARCHAR(255) NOT NULL,
  travel_time VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NULL,
  updated_at TIMESTAMP NULL
);

-- Table crew
CREATE TABLE crew (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name_fr VARCHAR(255) NOT NULL,
  name_en VARCHAR(255) NOT NULL,
  role_fr VARCHAR(255) NOT NULL,
  role_en VARCHAR(255) NOT NULL,
  bio_fr TEXT NOT NULL,
  bio_en TEXT NOT NULL,
  image VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NULL,
  updated_at TIMESTAMP NULL
);

-- Table technologies
CREATE TABLE technologies (
  id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  name_fr VARCHAR(255) NOT NULL,
  name_en VARCHAR(255) NOT NULL,
  description_fr TEXT NOT NULL,
  description_en TEXT NOT NULL,
  image_landscape VARCHAR(255) NOT NULL,
  image_portrait VARCHAR(255) NOT NULL,
  created_at TIMESTAMP NULL,
  updated_at TIMESTAMP NULL
);

-- Tables Spatie Permission
-- roles, permissions, model_has_roles, model_has_permissions, role_has_permissions

```

B. Routes Principales

```
// Routes publiques
GET / -> HomeController@index
GET /destination -> DestinationController@index
GET /crew -> CrewController@index
GET /technology -> TechnologyController@index
GET /language/{locale} -> LanguageController@switch

// Routes authentication (Breeze)
GET /login -> Auth\LoginController@showLoginForm
POST /login -> Auth\LoginController@login
POST /logout -> Auth\LoginController@logout
GET /register -> Auth\RegisterController@showRegistrationForm
POST /register -> Auth\RegisterController@register

// Routes admin (protégées par auth + role:admin)
GET /admin/planets -> Admin\PlanetController@index
GET /admin/planets/create -> Admin\PlanetController@create
POST /admin/planets -> Admin\PlanetController@store
GET /admin/planets/{id}/edit -> Admin\PlanetController@edit
PUT /admin/planets/{id} -> Admin\PlanetController@update
DELETE /admin/planets/{id} -> Admin\PlanetController@destroy

GET /admin/crew -> Admin\CrewController@index
// ... (même structure pour crew, technologies, users)
```

C. Permissions et Rôles

```
// Permissions
'planets.view', 'planets.create', 'planets.edit', 'planets.delete'
'crew.view', 'crew.create', 'crew.edit', 'crew.delete'
'technologies.view', 'technologies.create', 'technologies.edit', 'technologies.delete'
'users.view', 'users.create', 'users.edit', 'users.delete'

// Rôles et leurs permissions
'admin' => [toutes les permissions]
'planet_manager' => ['planets.*']
'crew_manager' => ['crew.*']
'technology_manager' => ['technologies.*']
```

D. Commandes Artisan Utilisées

```
# Initialisation
composer install
npm install
php artisan key:generate
php artisan storage:link

# Base de données
php artisan migrate
php artisan db:seed
php artisan migrate:fresh --seed

# Création de ressources
php artisan make:model Planet -m
php artisan make:controller Admin/PlanetController --resource
php artisan make:seeder PlanetSeeder
php artisan make:migration create_planets_table

# Tests
php artisan test
php artisan test --filter RouteTest

# Cache
php artisan view:clear
php artisan cache:clear
php artisan config:clear

# Build assets
npm run dev
npm run build
```

Fin du Rapport

Document généré le 26 Novembre 2025

Projet Space Tourism - Version 1.0